MAKERERE UNIVERSITY

COLLEGE OF COMPUTING AND INFORMATION SCIENCES

FINAL IMPLEMENTATION AND EVALUATION REPORT

Offline-First Neonatal Vital Sign Monitor with AI-Assisted Early Warning

Course: CSC 2118 - Embedded and Real-Time Systems

Prepared by GROUP D

Report revision: 2 November 2026

Table of Contents

CHAPTER 1: INTRODUCTION

1.1 Background

1.2 Problem Statement

1.3 Main Objective

1.3.1 Specific Objectives

1.4 Significance

1.5 Scope and Limitations

CHAPTER 2: LITERATURE REVIEW

CHAPTER 3: METHODOLOGY AND IMPLEMENTATION

3.1 Overview

3.2 Hardware Design

3.3 Software Development

3.4 Testing

3.5 Evaluation

3.6 Results by Specific Objective

3.7 Budget and Timeline

3.8 Ethical Considerations

CHAPTER 4: CONCLUSION AND RECOMMENDATIONS

REFERENCES

CHAPTER 1: INTRODUCTION

1.1 Background

Neonatal mortality remained a significant public-health challenge in Uganda and sub-Saharan Africa. Hypothermia and hypoxia required continuous observation, yet rural facilities often operated with limited staffing, intermittent electricity and unreliable connectivity. The project therefore implemented a low-cost, offline-first monitor that combined pulse oximetry, non-contact temperature measurement, local alarms, local storage and optional backend analysis.

1.2 Problem Statement

Rural clinics lacked affordable multi-parameter monitoring that continued to alert staff when Wi-Fi or cloud services were unavailable. The implemented system addressed this gap by keeping threshold alerts and data buffering on the ESP32-S3, while using the backend for history, risk scoring and dashboard visualization when connectivity returned.

1.3 Main Objective

The project designed, developed, tested and evaluated a low-cost offline-capable neonatal monitoring prototype that generated real-time local alerts for abnormal temperature and oxygenation, buffered readings locally, synchronized them when connectivity returned, and provided backend-assisted risk scoring and trend views.

1.3.1 Specific Objectives

Hardware design: The sensing, alert, display, timekeeping, storage and power interfaces were designed around the ESP32-S3. The tested display implementation used the ST7789 SPI screen; the earlier proposal LCD was replaced during implementation.

Software development: Firmware, API, PostgreSQL data storage, the explainable AI prototype and React dashboard were implemented as one connected workflow.

Testing: Hardware diagnostics, firmware builds, SD operations, API flows and end-to-end data handling were tested individually and together.

Evaluation: Recorded test evidence was compared with the proposed performance measures, while AI outputs were treated as synthetic research estimates rather than clinical diagnoses.

CHAPTER 2: LITERATURE REVIEW

The proposal established the need for continuous monitoring, offline operation and explainable warning. The implementation retained those requirements and made the safety boundary explicit: threshold alarms ran independently of the experimental model, and AI outputs were labelled synthetic and non-clinical. The final system therefore addressed the identified gap without treating a prototype model as a validated clinical instrument.

CHAPTER 3: METHODOLOGY AND IMPLEMENTATION

3.1 Overview of the Approach

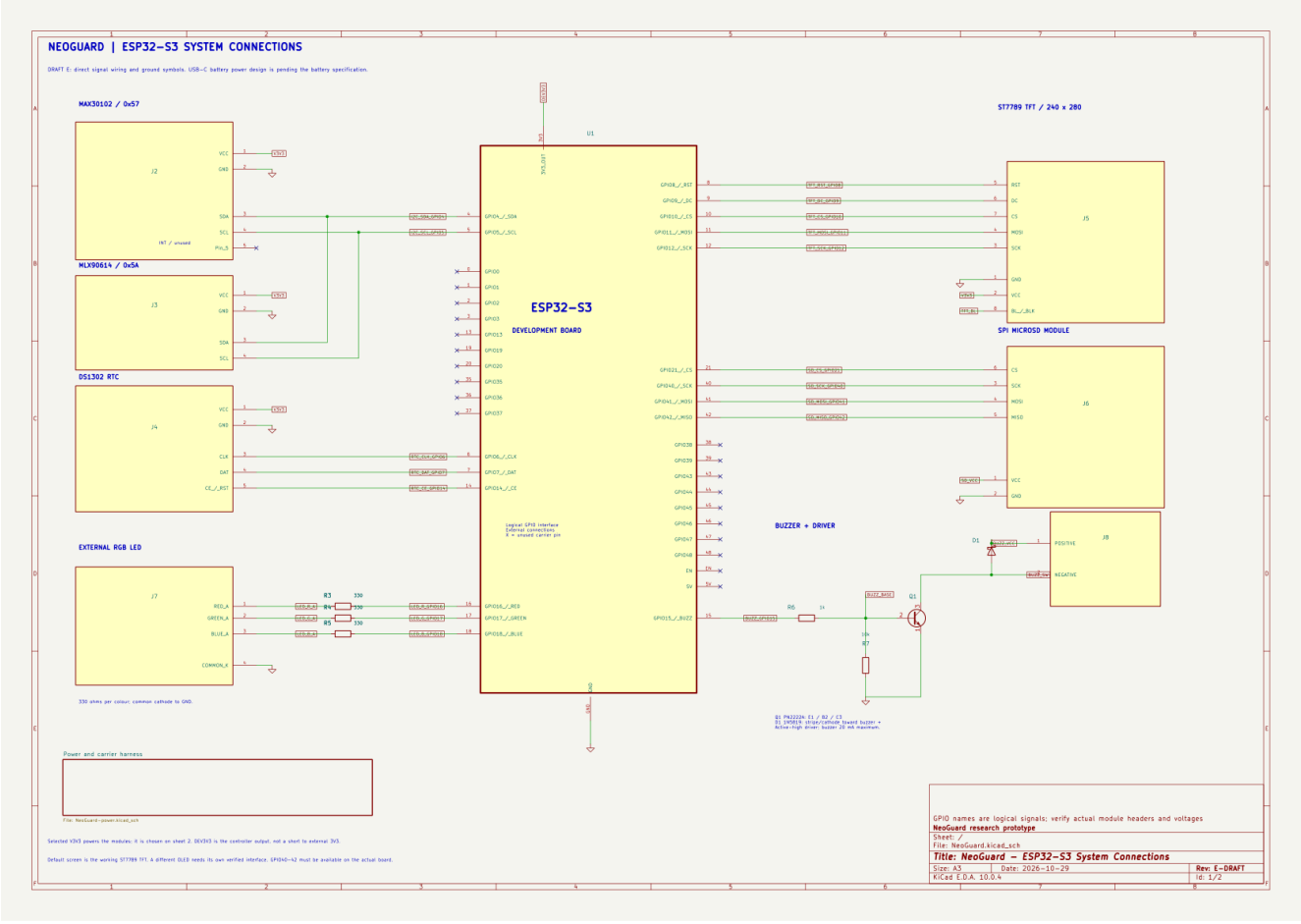
The work followed the Design, Develop, Test and Evaluate sequence from the proposal. Hardware connections were represented in a KiCad schematic before routing. Firmware was tested on the ESP32-S3, backend routes stored readings and alerts, the AI package trained on simulated data, and the dashboard consumed API responses rather than frontend sample data.

3.2 Hardware Design

The MAX30102 and MLX90614 shared the 3.3 V I2C bus on GPIO4 and GPIO5. The DS1302 used GPIO6, GPIO7 and GPIO14. The ST7789 used GPIO8, GPIO9, GPIO10, GPIO11 and GPIO12. The SD interface used GPIO21, GPIO40, GPIO41 and GPIO42. GPIO15 drove the buzzer driver; GPIO16, GPIO17 and GPIO18 drove the RGB channels through 330 ohm resistors. The schematic connected sensor power and returns to common ground and marked unused ESP32-S3 connections with no-connect symbols.

The routed PCB used two copper layers, filled ground pours and stitching vias. The later onboard ESP32-S3 and USB-C/LiPo power design was documented as a separate schematic revision so it could be reviewed without overwriting the validated carrier board.

Figure 3.1. Connected ESP32-S3 schematic with sensor, display, SD, RGB and buzzer wiring.



3.3 Software Development

The firmware sampled the sensors, applied local threshold logic, displayed status on the ST7789, drove the buzzer and RGB outputs, logged to SD and queued readings for later synchronization. The backend exposed authenticated REST endpoints, persisted patients, sessions, readings, alerts and device health in PostgreSQL, and enforced idempotent device-event handling. The React dashboard obtained KPI values and trends from the API.

The AI pipeline generated simulated monitoring sessions, trained an exploratory model and stored model/report artifacts. The backend could load the model in prototype mode, but threshold alerts remained independent of model availability. This separation prevented an unavailable or experimental AI component from disabling local safety alerts.

3.4 Testing

The firmware evidence recorded successful compilation and upload for the ESP32-S3. The dedicated RGB and SD diagnostic recorded four passing operations, including repeated write/read/append/compare/delete checks. The display and sensor diagnostic reported a partial pass: the display operated and the firmware ran, while heart-rate estimates remained unstable and accuracy was not established.

The repository also contained API and database tests for authentication, access control, patient/device/session creation, device synchronization, deduplication, stored history and alert handling. These results were used as software evidence; the hosted API itself was not reachable reliably during report generation, so no unavailable hosted metric was presented as a measured result.

3.5 Evaluation

The proposal specified sampling accuracy, sampling period and jitter, local alert latency, synchronization performance, AI false-alarm rate and detection lead time, backend response time, security checks, end-to-end delay, timekeeping drift and continuous operation. The available evidence supported implementation and integration claims, but it did not support clinical accuracy claims or a complete AI baseline comparison. The report therefore separated measured prototype checks from metrics that remained pending controlled data collection.

3.6 Results by Specific Objective

Specific objective	How it was obtained	Result and evidence
Hardware design	A KiCad schematic assigned every sensor, display, SD, RTC, alarm and RGB signal to named ESP32-S3 nets. The two-layer PCB was routed from that connectivity.	Achieved at prototype integration level. Schematic ERC and PCB DRC reports recorded zero violations, zero unrouted items and zero schematic-parity differences for the validated SPI carrier revision.
Software development	Firmware, FastAPI/Flask-compatible API services, PostgreSQL models, AI artifacts and React views were implemented. Offline readings were queued and synchronized by event identity.	Achieved as a working prototype. Repository tests and firmware logs documented successful build, upload, SD operations, API flow and dashboard data paths.
Testing	Separate diagnostics exercised display/sensors, RGB, SD, firmware build/upload, API behavior and integrated dataflow.	Partially achieved. Build/upload and SD diagnostics passed; display operation passed. Heart-rate stability and clinical accuracy remained unverified.
Evaluation	The project defined error, jitter, latency, synchronization, AI baseline and backend metrics and compared available evidence with those targets.	Partially achieved. Prototype implementation evidence was available; complete controlled AI false-alarm/lead-time and clinical accuracy datasets were not available.

The four objectives formed one chain: the hardware allocated reliable sensing and local alert paths; the software captured and interpreted those signals; testing verified the interfaces and dataflow; evaluation identified which performance claims were supported and which required further controlled experiments. Together, these stages produced the final offline-first monitoring prototype.

3.7 PCB RESULT

The 2D/3D PCB view showed the routed two-layer carrier with labelled ESP32 cable, sensors, RTC, SD, buzzer, RGB, power selectors and ST7789 display header. The later USB-C/LiPo and onboard-module concept remained a schematic revision pending exact component and battery dimensions.

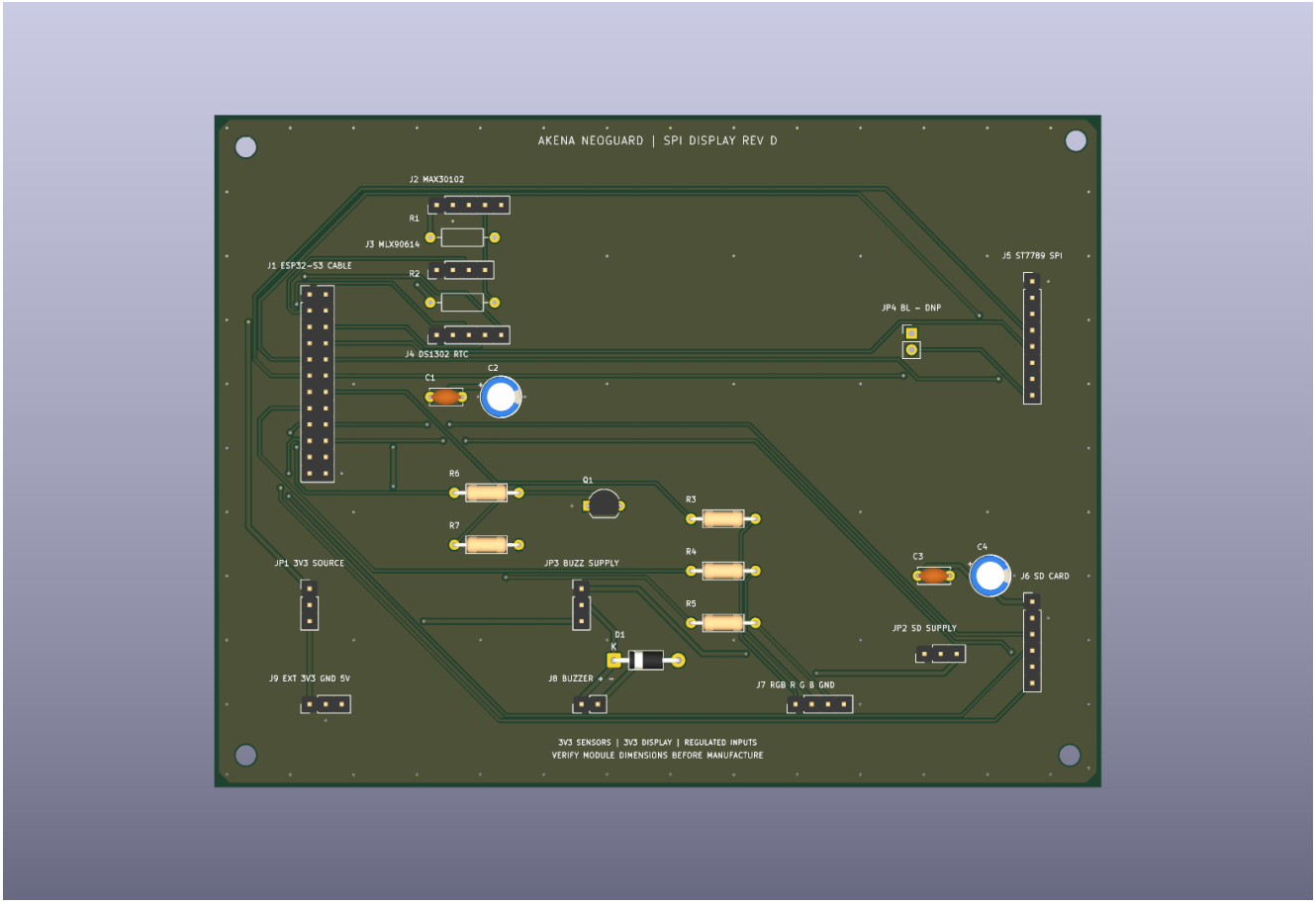


Figure 3.2. PCB layout/render used for the validated SPI carrier revision.

3.8 Budget and Cost Analysis

The proposal budget covered the ESP32, sensors, RTC, display, alarms, prototype wiring, USB power and optional SD storage. Implementation retained the low-cost module-based approach. The final onboard power revision added a USB-C connector, charger, LiPo cell and 3.3 V regulator, so the final procurement estimate required updating after the exact ESP32 module, battery capacity and display mechanics were selected.

3.9 Ethical Considerations and Limitations

Testing used student volunteers, bench checks and simulated data rather than neonates. The AI model was not clinically validated, did not constitute a diagnosis and was not presented as a clinical probability. Physical fit, sensor calibration, long-term battery safety, charge-current compatibility and hospital approval remained outside the demonstrated scope.

CHAPTER 4: CONCLUSION AND RECOMMENDATIONS

The project achieved a documented offline-first neonatal monitoring prototype. It connected the sensors and local alert devices to the ESP32-S3, implemented local storage and optional synchronization, provided API-backed dashboard views, and produced an explainable AI research path. The strongest evidence was the successful firmware build/upload, repeated SD diagnostic passes, connected schematic, routed PCB and zero reported KiCad rule/parity violations for the validated carrier revision.

Before clinical or manufacturing use, the system should be retested with calibrated reference instruments, a controlled battery and charger qualification, a confirmed ESP32-S3 module antenna layout, measured sampling/latency datasets, and an independently reviewed AI evaluation dataset.

REFERENCES

The references from the proposal were retained. Additional implementation references included Espressif ESP32-S3 hardware design guidance, the Texas Instruments BQ24074 datasheet for 1-cell power-path charging, and the TPS63001 datasheet for single-cell buck-boost regulation.

PERSONAL MOTIVATION AND FUTURE LEARNING IN FRANCE

This project represented more than a classroom prototype: it was a practical demonstration of my passion for artificial intelligence, embedded systems and socially useful engineering. I wanted to build a device that could continue working in resource-limited settings, while also learning how data, software and hardware could be combined responsibly. Studying in France would allow me to deepen this foundation through advanced courses, laboratories and collaboration in embedded computing, machine learning, signal processing, medical technology and responsible AI. I would bring the discipline developed through this project—careful testing, transparent limitations and a willingness to learn—and use it to contribute to research and engineering teams.

The project also showed me what I still need to master: calibrated biomedical sensing, robust model validation, low-power electronics, secure connected systems and deployment with clinicians. France would give me the academic environment and mentorship to turn this prototype into a more rigorous, safe and generalisable system.

FULL SYSTEM ARCHITECTURE

The architecture below was used to explain how bedside sensing, the backend API, the dashboard and the AI risk-modeling components work together.

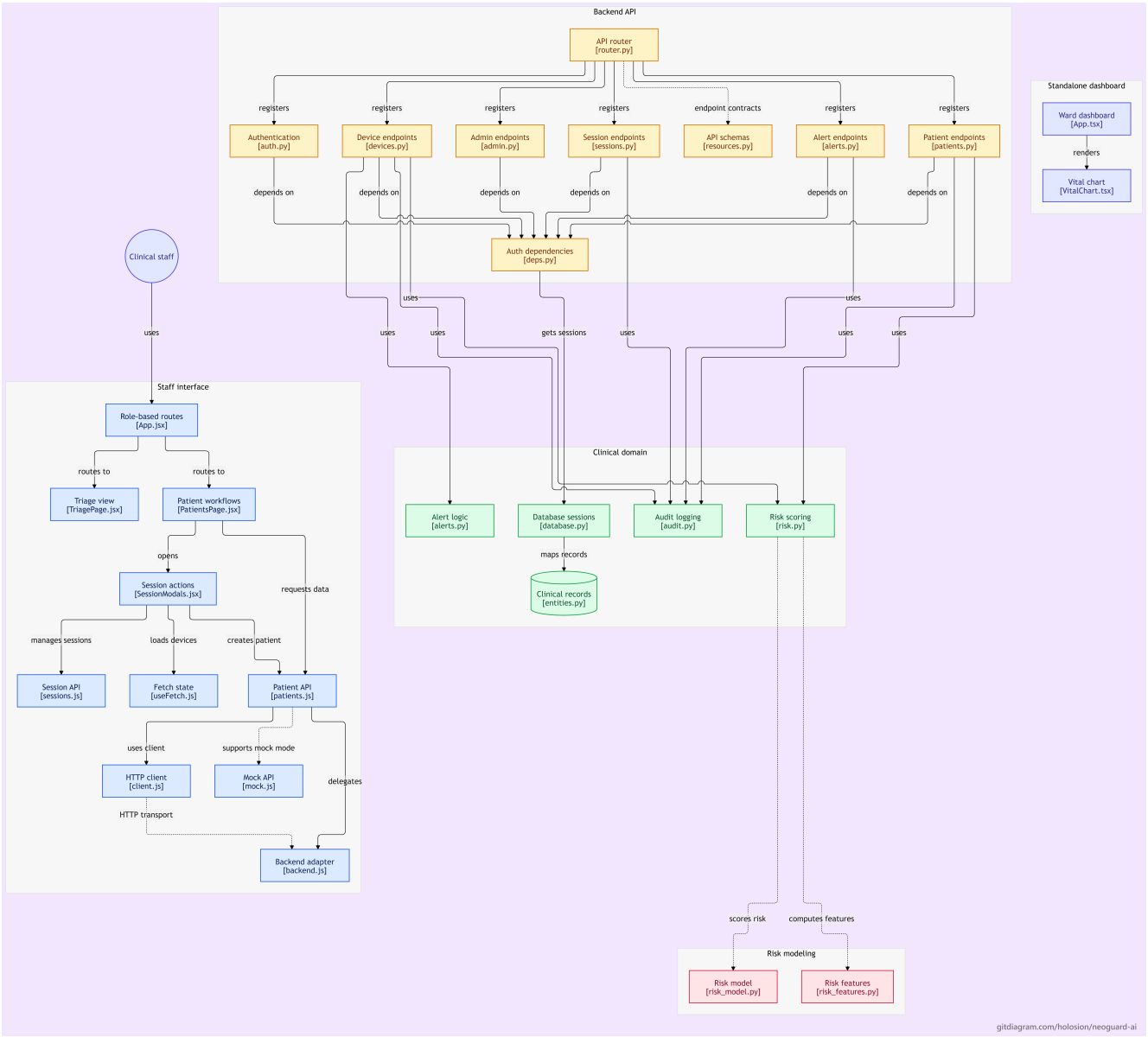


Figure 4.1. Full NeoGuard application and AI architecture supplied for the pitch report.